Memory Management

myposter Academy

Lukas Rieger

Memory Management

Why memory management matters

Every program allocates dynamic memory on the **heap**. Allocation is cheap; the hard part is knowing **when to free**.

```
free(buf);
// later...
printf("%s", buf); // undefined!
```

Too early: **dangling pointer** (crash, CVE).

Manual management is error-prone [1]: liveness is *global*, but `free` is **local**.

```
void process() {
  char *tmp = malloc(1024);
  if (error) return; // leaked!
  free(tmp);
}
```

Too late: **memory leak** (exhaustion).

Solution: delegate freeing to an **automatic collector**.

What we'll cover

Three approaches to automatic memory management, each with different trade-offs:

Tracing GC
Java / Kotlin

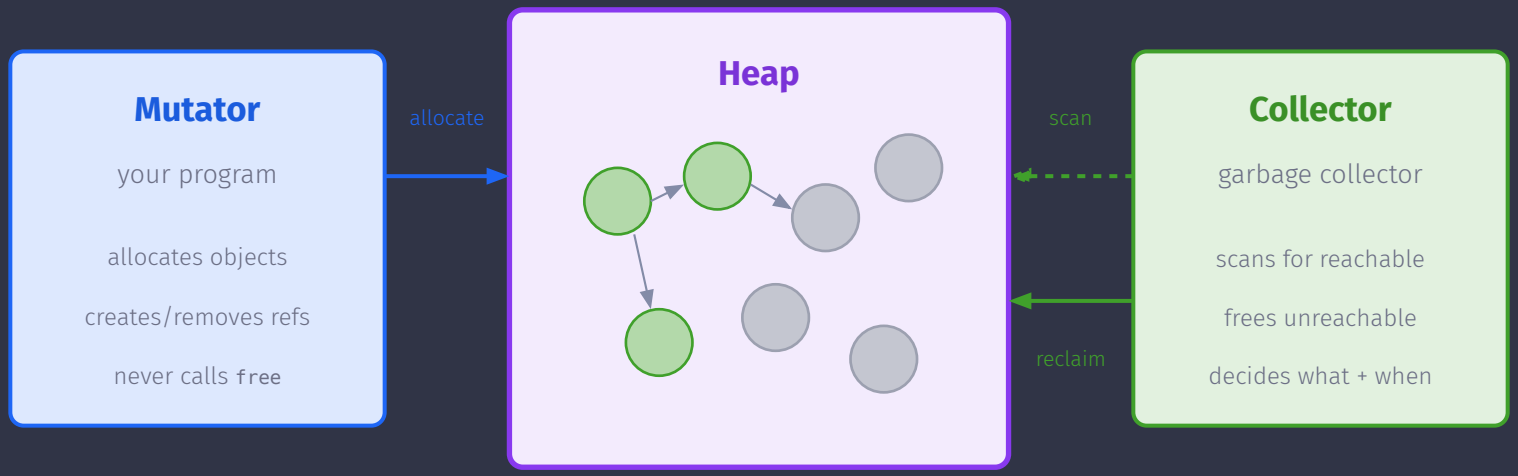
Ref Counting
Swift / ObjC

Ownership
Rust

We'll explore each, compare them, and see why different platforms made different choices.

Tracing Garbage Collection

The mutator and the collector



Two actors share the heap. The **mutator** (your program) allocates objects and manipulates references but never frees memory. A separate **collector** periodically identifies unreachable objects and reclaims their memory. The programmer writes normal code; the collector handles deallocation [2].

Mark-and-Sweep

Foundational tracing algorithm [3]:

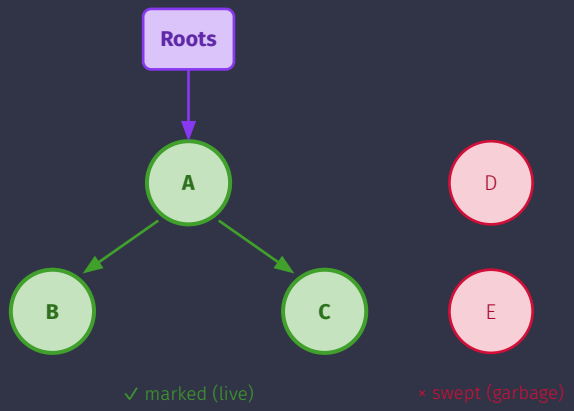
Mark phase

Trace from **roots** (stack, globals), mark all reachable objects.

Sweep phase

Walk entire heap, reclaim everything **unmarked**.

Simple, correct, but requires **stopping the mutator** for the entire duration.



Where pauses come from

Basic mark-and-sweep = **stop-the-world** (STW):

- Mutator halted for entire mark + sweep duration
- Pause proportional to **heap size**, not garbage
- Large heaps = long pauses = visible jank

This is why “GC is slow” became folklore. But modern collectors don’t work this way.

Parallel and concurrent collection

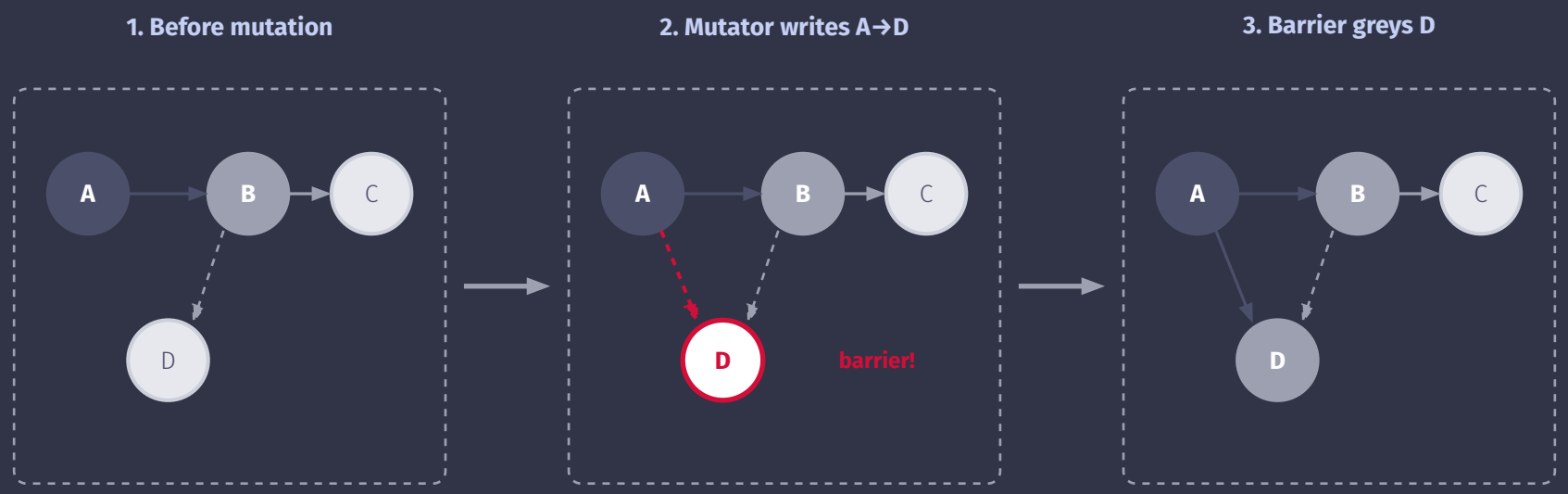
Parallel collection

- Multiple collector threads work **simultaneously**
- Mutator still **stopped** during collection
- Faster STW pauses (work split across cores)
- Simpler: no mutator interference

Concurrent collection

- Collector and mutator run **at the same time**
- **Write barriers** track mutator changes during marking
- Much shorter (or no) STW pauses
- Trade-off: more CPU overhead, more complexity

The tricolour invariant

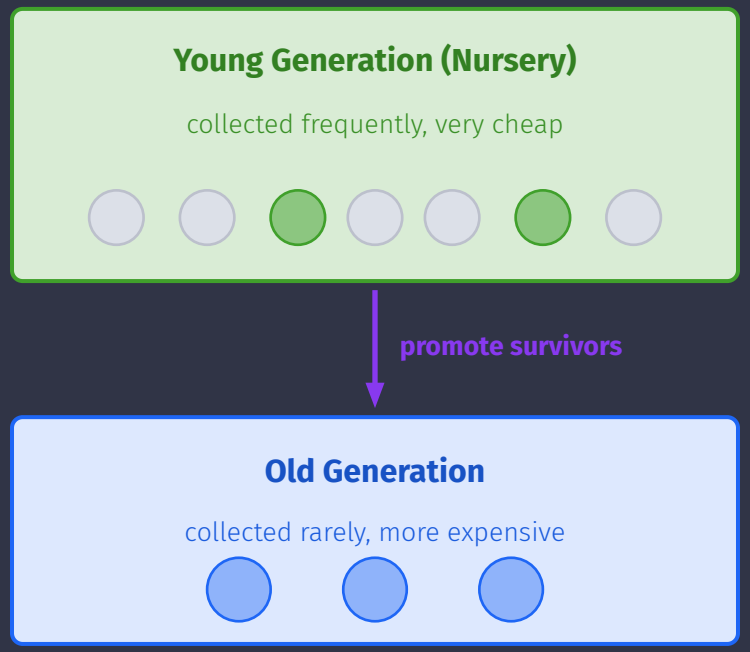


The tricolour abstraction [2]: **Black** (fully scanned), **Grey** (children pending), **White** (not yet visited).

Key invariant: no black → white without grey in between. Write barriers enforce this, which is what makes **concurrent** collection possible.

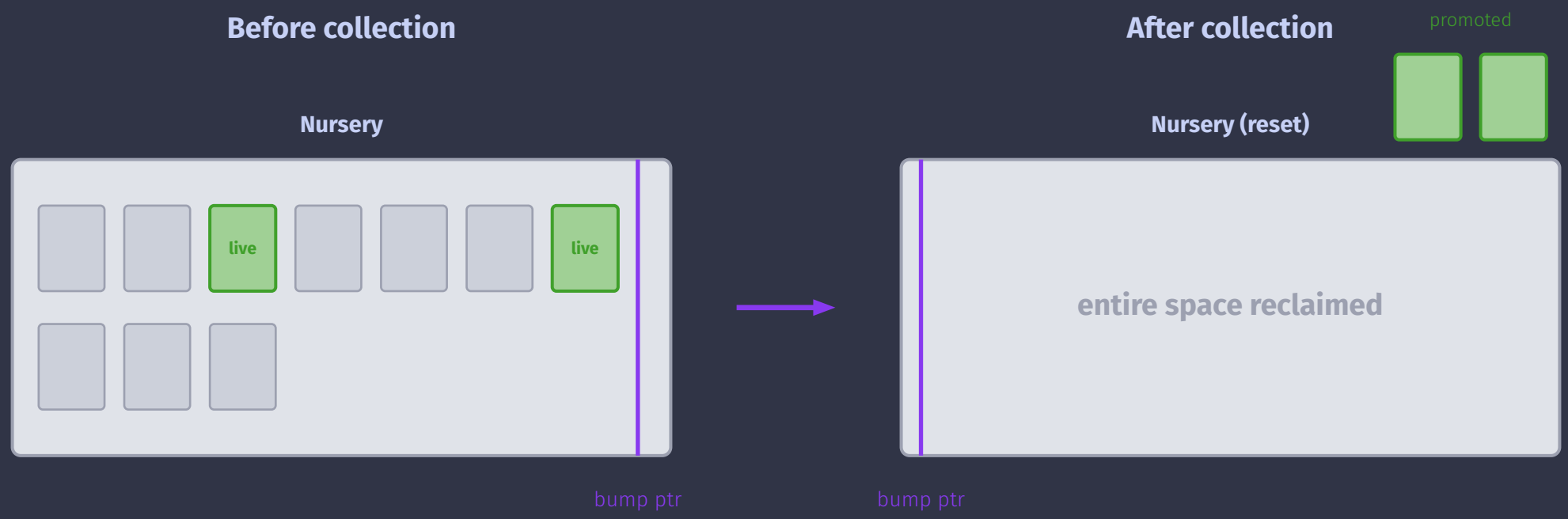
Generational Garbage Collection

Empirical observation (**weak generational hypothesis**): in virtually all programs, the vast majority of objects die shortly after allocation [2]. This motivates splitting the heap by age:



Nursery collection: why it's fast

The nursery uses **bump-pointer allocation** (just increment a pointer) and is reclaimed **in bulk**. The collector never visits dead objects individually:



9 dead, 2 live. Collector **never touches** dead ones.

Nursery wiped. Bump ptr resets. **Zero per-object cost.**

The price of Garbage Collection

Garbage Collection needs **heap headroom** to perform well:

- Matches `malloc`/`free` throughput only at **5× min heap** [4]
- Typical overhead: ~ 17% [4]
- Typical config: **1.3–4× minimum heap** [2]

On **memory-constrained devices**, this headroom is a luxury.

⚠️ **Common Misconception**

"GC is slow." Naive STW mark-sweep is slow. Modern concurrent generational collectors avoid most pauses and reclaim nurseries in bulk at near-zero cost.

Garbage Collection on Android

ART (Android Runtime) is the VM that replaced Dalvik in Android 5.0. All Kotlin/Java apps run on it.

As of Android 14, ART has two production collectors [5]:

- **Concurrent Copying** collector (generational, default)
- **Concurrent Mark-Compact** collector (newer, reduces fragmentation)

Collection is **concurrent** but uses a single GC thread (**HeapTaskDaemon**), not parallel multi-threaded collection like server JVMs [5]. Lower bound on GC overhead: **2–51%** across real-world apps [5].

Android's memory challenges

Mobile GC faces unique constraints not seen on servers:

- **No memory overcommit:** Android sets fixed per-app heap budgets; no swapping [6]
- **Kill, don't swap:** under memory pressure, Android kills background apps. Restarting takes **4–27× longer** than reading from swap [6]
- **16 ms frame budget** at 60 Hz (less at 90/120 Hz): any GC pause = visible jank
- **Apps fight back:** popular apps (Instagram, Google Maps) introspect their own heap and adjust behavior based on available memory, creating feedback loops [5]

💡 **Did you know?**

Popular apps allocate large heaps (60–260 MB) but **actively access** only a fraction (< 30 MB). The rest is aggressively cached data; discarding it forces expensive network refetches [6].

Reference Counting

How reference counting works

Unlike tracing GC, RC has no separate collector. Instead, each object maintains a **reference count** in its header [2]:

- On pointer write: increment new target's rc, decrement old target's rc
- When rc drops to **zero**: free object and recursively decrement its children

```
var a: MyObject? = MyObject() // rc = 1
var b = a                     // rc = 2
b = nil                       // rc = 1
a = nil                       // rc = 0 → freed!
```

Advantages and trade-offs

Advantages

- **Deterministic:** freed when unreachable
- **Low memory overhead:** works in near-full heaps [2]
- **No collector pauses:** no separate GC phase, cost distributed across mutations

Trade-offs

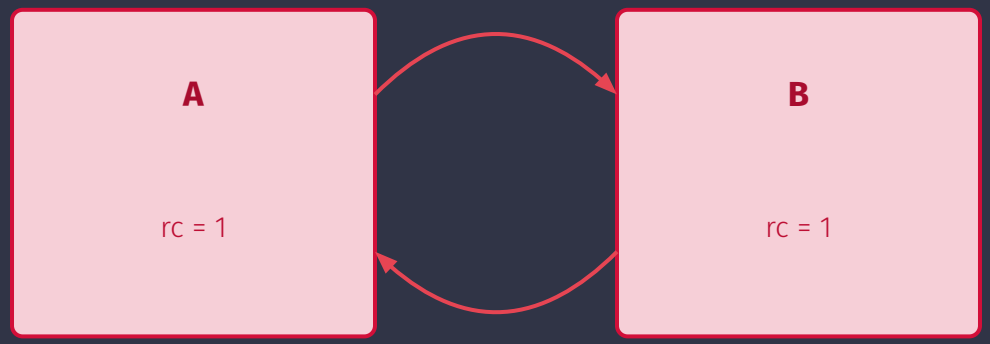
- Every pointer write = **read-modify-write**
- Cache pollution from refcount writes [2]
- Per-object header overhead
- **Cannot reclaim cycles**

⚠️ Common Misconception

"RC has no pauses." Cascading frees can spike **worse** than GC pauses [2].

The cycle problem

RC **cannot reclaim cycles**. Even if unreachable from roots, internal references keep rc > 0. Common: “doubly-linked lists, trees with back pointers” [2].



no external refs → unreachable, leaked

Breaking cycles in Swift

- `weak var`: doesn't increment **strong** rc, auto-zeroed to `nil` on dealloc
- `unowned let`: doesn't increment **strong** rc, checked at access, fatal error if gone

```
class Child {
    weak var parent: Parent?  // won't create a cycle
}
```

Why did iOS choose RC, Android GC?

iOS / Swift: ARC

- Early iPhones: 128–256 MB RAM, no room for GC headroom
- Objective-C already had manual retain/release
- ARC = compiler automates existing practice
- Deterministic cleanup critical for constrained devices

Android / Kotlin: Tracing GC

- JVM heritage: GC built into Dalvik/ART from the start
- Server-style heap management, generous memory budgets
- GC excels at short-lived objects (dominant in app workloads)
- Concurrent GC mitigates pause concerns

GC vs ARC: when to use which

Tracing GC works well when...

- High allocation rate, most objects short-lived
- Heap memory plentiful
- Developer productivity prioritized
- Cycles are common

Weak: headroom (1.3–4×), pauses, ~ 17% overhead [4]

RC works well when...

- Memory constrained (no headroom needed)
- Deterministic cleanup critical
- Latency predictability matters most
- Interop with manual memory management

Weak: per-write overhead, cache pollution, cycles [2]

💡 **Did you know?**

As both are optimized, GC and RC **converge** [7]. But both pay a runtime price...

Ownership & Borrowing

The third way: ownership

GC and RC both pay at runtime. What if the **type system** handled memory?

Rust's answer: each value has **one owner**. Assignment = **move**. Old binding invalid.

```rust
fn main() {
    let s = String::from("hello");  // s owns the String
    let t = s;                      // ownership moves to t
    // println!("{}", s);            // compile error: s is invalid
    println!("{}", t);               // OK
}   // t is dropped here, memory freed
```

Formal foundation: **affine types** (Girard, 1987). Values used **at most once**. No GC, no RC. Deallocation: **deterministic**, **scope-based**, **zero runtime cost**.

Borrowing

Move on every access = impractical.
Borrowing grants temporary access.

Shared borrows (`&T`)

Read-only. Multiple allowed.

```rust
fn len(s: &String) -> usize {
    s.len()
}
let s = String::from("hi");
println!("{}", len(&s)); // borrow
```

Mutable borrows (`&mut T`)

Read-write. Only **one** at a time.

```rust
fn push(s: &mut String) {
    s.push('!');
}
let mut s = String::from("hi");
push(&mut s);
println!("{s}"); // prints "hi!"
```

The rule: *one mutable* or *any number of shared*, never both.

Lifetimes and escape hatches

Lifetimes

Compiler tracks **lifetimes**: every reference must remain valid.

```
fn first_word(s: &str) -> &str {
    &s[..s.find(' ')
          .unwrap_or(s.len())]
}
```

Mostly inferred. Dangling pointers = **compile errors**.

The escape hatch

When you need **shared ownership**:

```
use std::rc::Rc;

let a = Rc::new(vec![1, 2, 3]);
let b = Rc::clone(&a); // rc = 2
```

`Rc<T>` / `Arc<T>` = reference counting, opt-in. Reintroduces cycle risk.

No free lunch

What you get

- Memory safety **without runtime overhead**
- No GC pauses, no refcount traffic
- No heap headroom tax
- Bugs surface at **compile time**

What you pay

- Borrow checker rejects valid-looking code
- Graphs, linked lists, observers need `unsafe` or smart pointers
- `Copy` = implicit memcpy (cheap for `i32`, costly for large structs)
- `Clone` makes duplication explicit, but adds verbosity
- Ownership analysis lengthens compile times

Wrapping Up

Lesser-known approaches

Arenas
Zig, Odin, Ada

Regions
Pony, Verona

Generational References
Vale

Hardware Safety
CHERI

Constraint References
Game engines

Interaction Nets
HVM

Neverfree
Missiles(!)

Stack Arenas
Basil

Linear Reference Counting
(theoretical)

📚 Further Reading

The Memory Safety Grimoire [8] catalogs **17+ distinct approaches**. The design space is far from settled.

Key takeaways

1. **Memory management is a design decision**, shaped by platform constraints
2. **Tracing GC:** throughput with short-lived objects, needs heap headroom
3. **Reference counting:** deterministic cleanup, low overhead, but cycles and per-write cost
4. **Ownership:** compile-time safety, zero runtime cost, steeper learning curve
5. GC and RC **converge** as both are optimized

Thank you! Questions?

References

[1] P. R. Wilson, “Uniprocessor Garbage Collection Techniques,” in *Proc. International Workshop on Memory Management (IWMM)*, 1992.

[2] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2012.

[3] J. McCarthy, “Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I,” *Communications of the ACM*, vol. 3, no. 4, 1960.

[4] M. Hertz and E. D. Berger, “Quantifying the Performance of Garbage Collection vs. Explicit Memory Management,” in *Proc. ACM OOPSLA*, 2005.

[5] K. Sareen, S. M. Blackburn, S. S. Hamouda, and L. Gidra, “Memory Management on Mobile Devices,” in *Proc. ACM SIGPLAN ISMM*, 2024.

[6] N. Lebeck, A. Krishnamurthy, H. M. Levy, and I. Zhang, “End the Senseless Killing: Improving Memory Management for Mobile Operating Systems,” in *Proc. USENIX ATC*, 2020.

[7] D. F. Bacon, P. Cheng, and V. T. Rajan, “A Unified Theory of Garbage Collection,” in *Proc. ACM OOPSLA*, 2004.

[8] E. Ovadia, “The Memory Safety Grimoire.” [Online]. Available: https://verdagon.dev/grimoire/grimoire